Synaptic — Master Architecture

Synaptic — Master Architecture & Developer Reference

Version: Production v4 — March 2026 (Multi-TF HMM + Athena LLM) **Deployment:** Railway (Docker, PostgreSQL) **Exchanges:** CoinDCX (live), Binance (paper/testnet data source)

Table of Contents

1. System Overview
2. High-Level Architecture
3. Python Trading Engine
    - 3.1 Main Bot Loop (main.py)
    - 3.2 Data Pipeline (data_pipeline.py)
    - 3.3 Feature Engine (feature_engine.py)
    - 3.4 HMM Brain (hmm_brain.py)
    - 3.5 Segment Features (segment_features.py)
    - 3.6 Coin Scanner (coin_scanner.py)
    - 3.7 Risk Manager (risk_manager.py)
    - 3.8 Execution Engine (execution_engine.py)
    - 3.9 Tradebook (tradebook.py)
    - 3.10 Athena LLM Layer (llm_reasoning.py)
    - 3.11 Engine API (engine_api.py)
4. Signal Pipeline — End-to-End Flow
5. Exit Mechanics
6. Risk Management
7. Crypto Segments
8. SaaS Dashboard (NextJS)
    - 8.1 Technology Stack
    - 8.2 Application Pages
    - 8.3 API Routes — Complete List
    - 8.4 Bot Lifecycle
    - 8.5 Auto Re-Registration
    - 8.6 Trade Sync Flow
    - 8.7 BTC Confidence Display
9. Data Routing — End-to-End
10. Deployment & Infrastructure
11. Configuration Reference

---

1. System Overview

Synaptic (Project Regime-Master) is an automated cryptocurrency futures trading system. It uses Hidden Markov Models (HMM) on multiple timeframes to classify market regimes (Bull/Bear/Chop), then routes signals through an LLM reasoning layer (Athena) before executing trades.

The system is multi-tenant: multiple users can run isolated bots simultaneously on a shared engine. Each bot is scoped to a market segment (L1, L2, AI, etc.) and executes trades with its own `bot_id` stamp for full trade isolation.

Who it's for: Internal use — the engine operator deploys bots for end-users via the SaaS dashboard.

What it does: - Every 15 minutes: scans a segment-based coin pool, runs 3-TF HMM analysis per coin, computes conviction scores, calls Athena for final LLM validation, and deploys limit or market orders - Every 10 seconds (heartbeat): updates unrealized P&L, manages trailing SL steps, checks exit conditions, handles limit order expiry/fill - Persists all state to JSON files (engine) and PostgreSQL (SaaS dashboard)

---

2. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                        Railway Platform                          │
│                                                                   │
│  ┌──────────────────────────┐    ┌──────────────────────────┐    │
│  │   Python Trading Engine  │    │   NextJS SaaS Dashboard  │    │
│  │   (Flask + main loop)    │    │   (Next.js 14 App Router)│    │
│  │                          │    │                          │    │
│  │  engine_api.py (Flask)   │◄───│  /api/bot-state GET      │    │
│  │  └─ main.py (bot loop)   │    │  /api/bots/toggle POST   │    │
│  │     ├─ hmm_brain.py      │    │  /api/trades/* POST      │    │
│  │     ├─ feature_engine.py │    │                          │    │
│  │     ├─ coin_scanner.py   │    │  Prisma ORM → PostgreSQL │    │
│  │     ├─ risk_manager.py   │    │  NextAuth sessions       │    │
│  │     ├─ llm_reasoning.py  │    │  Subscription gating     │    │
│  │     ├─ tradebook.py      │    └──────────────────────────┘    │
│  │     └─ execution_engine  │                                    │
│  │                          │    ┌──────────────────────────┐    │
│  │  Data: data/*.json       │    │   PostgreSQL Database    │    │
│  └──────────────────────────┘    │   User / Bot / Trade     │    │
│                                  │   BotSession / CycleSnap │    │
│  ┌──────────────────────────┐    └──────────────────────────┘    │
│  │  External APIs           │                                    │
│  │  CoinDCX (live orders)   │                                    │
│  │  Binance (OHLCV data)    │                                    │
```

```
│   │  Gemini API (Athena LLM) │                                  │
│   └──────────────────────────┘                                  │
└─────────────────────────────────────────────────────────────────┘
```

Two-layer architecture: - **Python Engine** — stateful trading loop, market analysis, order execution, tradebook. Exposes a Flask REST API. State persisted in `data/*.json`. - **NextJS SaaS** — user-facing dashboard, bot management, trade display, subscription. Reads engine state via HTTP. Writes user/bot/trade records to PostgreSQL.

The two layers communicate via HTTP only. The engine has no direct DB access; the SaaS dashboard has no direct market data access.

---

3. Python Trading Engine

3.1 Main Bot Loop (main.py)

Entry point: `engine_api.py` starts `RegimeMasterBot` in a background thread via `_run_engine()`.

Two timing loops:

Loop	Interval	Purpose
Heartbeat (`_heartbeat`)	10 seconds (`LOOP_INTERVAL_SECONDS`)	Trailing SL sync, limit order management, P&L updates, pause/halt checks
Analysis Cycle (`_tick`)	15 minutes (`ANALYSIS_INTERVAL_SECONDS`)	Full coin scan, HMM analysis, conviction scoring, Athena call, trade deployment

`_tick()` — Full Analysis Cycle steps:

1. Check weekly tier re-classification (background thread if due)
2. Reset Athena rate limiter for the new cycle
3. Refresh segment heatmap JSON (cheap ticker call)
4. Rebuild coin pool from `get_active_bot_segment_pool(ENGINE_ACTIVE_BOTS)` every `SCAN_INTERVAL_CYCLES` cycles
5. Rotate through pool in batches; skip already-deployed coins
6. Fetch live CoinDCX prices (funding rates, etc.)
7. Fetch balance; retry up to 3x in live mode if $0
8. Check kill switch (10% drawdown in 24h)
9. Run `_check_exits()` — syncs `_active_positions` dict from tradebook (regime-change exits disabled)
10. BTC flash-crash macro veto (blocks all BUY signals if BTC drops >1.5% in 15m)
11. `_analyze_coin()` for each symbol → builds `raw_results` list
12. Deploy loop — iterates per registered bot in `ENGINE_ACTIVE_BOTS`:

    - Filter raw_results to bot's segment
    - Pick top `TOP_COINS_PER_SEGMENT` (=1) coin by conviction
    - Skip if conviction < `MIN_CONVICTION_FOR_DEPLOY` (60)
    - Skip if `bot_id:symbol` already in active tradebook keys (duplicate guard)
    - Call Athena — VETO or EXECUTE
    - Execute trade, stamp with `bot_id`
    - Record in tradebook via `tradebook.open_trade()`
13. Save multi-state JSON for dashboard
14. POST cycle snapshot to dashboard `/api/cycle-snapshot` (background thread)

`_heartbeat()` tasks: - Read `engine_state.json` — skip if paused/halted - Process commands file (KILL/CLOSE_ALL) - `_sync_positions()` — remove closed trades from `_active_positions` - `_manage_limit_orders()` — TIF expiry, escape hatch, paper fills, virtual limit triggers - `tradebook.update_unrealized()` — P&L updates, trailing SL stepping, SL/TP exit checks (paper) - Live mode: sync CoinDCX positions, sync live TP/SL

Key in-memory state:

```
_multi_tf_brains: dict[symbol → MultiTFHMMBrain]   # 3-TF brains per coin
_coin_brains:     dict[tf_key → HMMBrain]           # per-coin-per-TF brain cache
_coin_states:     dict[symbol → dict]               # dashboard state per coin
_active_positions: dict[bot_id:symbol → dict]       # active position tracker
_live_prices:     dict[cdx_pair → dict]             # live funding/price data
```

Segment inference fallback: `_infer_segment_from_name(bot_name)` maps bot name keywords → segment when `segment_filter` is not in registration payload.

Signal broadcast audit log: Every signal event (FILTERED, ATHENA_DECISION, SIGNAL_DISPATCH, TRADEBOOK_RECORDED) is written to `data/signal_broadcast.log` via `_bcast()`. Rotates daily, 7-day retention.

3.2 Data Pipeline (data_pipeline.py)

Unified OHLCV data layer. Always tries CoinDCX first, falls back to Binance.

Function	Description
`fetch_klines(symbol, interval, limit)`	Primary kline fetcher. CoinDCX → Binance fallback
`fetch_futures_klines(symbol, interval, limit)`	Futures klines. Paper→Binance Futures, Live→CoinDCX
`get_multi_timeframe_data(symbol)`	Fetches 15m, 1h, 4h simultaneously
`get_current_price(symbol)`	Live price. Always CoinDCX, Binance fallback
`_get_binance_client()`	Lazy singleton Binance client

Supported intervals: 1m, 3m, 5m, 15m, 30m, 1h, 4h, 1d, 1w

The `_parse_klines_df()` function normalizes raw kline arrays to a clean OHLCV DataFrame with `timestamp` (datetime), `open`, `high`, `low`, `close`, `volume` columns.

3.3 Feature Engine (feature_engine.py)

Two-pass computation:

Pass 1 — HMM Features (`compute_hmm_features(df, btc_df=None)`):

Feature	Description
`log_return`	Log(close/close.shift(1))
`volatility`	(high - low) / close
`volume_change`	Log(volume/volume.shift), clipped ±3
`vol_zscore`	Z-score of volume vs 24-period SMA, clipped ±5
`rel_strength_btc`	Asset return - BTC return (requires btc_df); jittered to prevent singular covariance
`liquidity_vacuum`	Abs log-return / (ATR14/price), clipped 0-5
`exhaustion_tail`	Wick skew × vol_zscore (captures rejection candles)
`amihud_illiquidity`	Abs log-return / dollar volume × 1e8, clipped 0-10
`volume_trend_intensity`	EMA5/EMA20 of volume (momentum of volume)
`swing_l`	10-candle rolling min of low (for RM3_Swing SL)
`swing_h`	10-candle rolling max of high (for RM3_Swing SL)

Pass 2 — Technical Indicators (`compute_indicators(df)`): Adds `rsi`, `bb_upper`, `bb_middle`, `bb_lower`, `atr`.

Convenience: `compute_all_features(df)` runs both passes.

Other exported functions: - `compute_ema(series, length)` — EWM with adjust=False - `compute_trend(df)` → 'UP'/'DOWN'/'FLAT' via EMA 20/50 crossover - `compute_atr(df, length=14)` — EWM ATR - `compute_support_resistance(df)` → dict with support/resistance/pivot/bb_pos - `compute_sr_position(df)` → (sr_position, vwap_position) for conviction scoring - `compute_vwap(df, window=20)` — rolling VWAP

3.4 HMM Brain (hmm_brain.py)

`HMMBrain` — Single coin, single timeframe

Model: `GaussianHMM` from hmmlearn. `n_components=3` (HMM_N_STATES), `covariance_type="full"`, 100 iterations.

Training (`train(df)`): 1. Extracts coin's specific feature list from `segment_features.py` 2. Z-score normalizes features (stores mean/std for prediction scaling) 3. Fits GaussianHMM 4. Calls `_build_state_map()` to assign canonical regime labels

`_build_state_map()`: - Finds `log_return` index in `self.features` dynamically (NOT hardcoded 0) - Finds `volatility` index in `self.features` dynamically (NOT hardcoded 1) - Sorts raw HMM states by mean log-return (descending) - Maps: highest return → BULL (0), middle → CHOP (2), lowest → BEAR (1) - For 4-state models: uses volatility to distinguish CHOP from BEAR among middle states

Prediction (`predict(df)`): - Returns `(canonical_state: int, confidence: float)` - Confidence = **margin** = best_prob - 2nd_best_prob (range 0.0–1.0) - Raw max-posterior was always 99%+ regardless of accuracy; margin measures decisiveness

Retrain trigger: `needs_retrain()` returns True if model is untrained or age > `HMM_RETRAIN_HOURS` (24h)

`MultiTFHMMBrain` — Multi-timeframe aggregator

Manages 3 separate `HMMBrain` instances per coin (1d, 1h, 15m).

Conviction computation (`get_conviction()`):

```
Timeframe weights: 1d=40%, 1h=35%, 15m=25%

For each TF with a non-CHOP prediction:
  Direction vote: BULL→BUY, BEAR→SELL, CHOP→no vote

Consensus = majority of non-CHOP votes
  If tied: return 0, None, 0

Weighted score per TF (if agrees with consensus):
  margin >= 0.30 → full weight (100%)
  margin >= 0.20 → 85% weight
  margin >= 0.10 → 65% weight
  margin >= 0.05 → 40% weight
  below 0.05    → 20% weight

Disagreement = 0 (no penalty, no contribution)
CHOP = 0

conviction = sum(w × tier_multiplier), clipped 0–100
```

Minimum agreement required: 2 of 3 TFs (`MULTI_TF_MIN_AGREEMENT`).

Regime summary format (used in dashboard): `"1d=BULLISH(0.45) | 1h=BULLISH(0.32) | 15m=SIDEWAYS/CHOP(0.08)"`

3.5 Segment Features (segment_features.py)

Per-coin feature lists determined via 15m Permutation Likelihood backtesting across 8 segments and 4 timeframes.

Global default: `ALL_HMM_FEATURES` — 9 features used for unknown/unmapped coins.

`COIN_FEATURES` **dict:** 41 coins with individually optimized feature subsets. Examples:

Coin	Features
BTCUSDT	vol_zscore, log_return, volume_trend_intensity, liquidity_vacuum, amihud_illiquidity, exhaustion_tail, volatility, volume_change
SOLUSDT	vol_zscore, liquidity_vacuum, amihud_illiquidity, volume_trend_intensity, exhaustion_tail, log_return, rel_strength_btc
TAOUSDT	vol_zscore, liquidity_vacuum, log_return, rel_strength_btc, amihud_illiquidity, volume_trend_intensity, exhaustion_tail

Note: BTCUSDT includes `log_return` explicitly because `_build_state_map()` searches for it by name — this is required for correct state ordering.

API: - `get_features_for_coin(coin)` → returns coin-specific list or ALL_HMM_FEATURES fallback - `get_segment_for_coin(coin)` → returns segment name from `config.CRYPTO_SEGMENTS`, defaults to "L1"

3.6 Coin Scanner (coin_scanner.py)

Scan Pool Building

`get_active_bot_segment_pool(active_bots)` — called each cycle in `_tick()`:

1. Inspect `segment_filter` for each active bot
2. If any bot has `segment_filter="ALL"` or no bots registered → trigger dynamic segment selection
3. Dynamic mode: call `get_hottest_segments(SEGMENT_SCAN_LIMIT=3)` → writes `data/segment_heatmap.json`
4. Compile coins from all target segments
5. Apply static exclusions (`COIN_EXCLUDE` + `EXCLUDED_COINS`) and dynamic exclusions (`coin_exclusions.json`)

6. **Always insert PRIMARY_SYMBOL (BTCUSDT) at index 0** — required for BTC macro context and dashboard display

Institutional Segment Heatmap (`get_hottest_segments`)

3-Pillar scoring for each segment:

Pillar	Formula	Weight
VW-RR (Volume-Weighted Relative Return)	Σ(coin_change × volume_weight)	Part of composite
Benchmark Alpha	VW-RR - BTC 24h return	Part of composite
Participation Breadth	% coins moving in segment direction	Multiplier

`composite_score = VW-RR × (breadth_pct / 100)`

Segments ranked by absolute composite score. Top `SEGMENT_SCAN_LIMIT` (3) returned.

Coin Tier System

Coins classified into Tier A/B/C via weekly calibration experiment (`tools/weekly_reclassify.py`): - Tier A: stable forward Sharpe ≥ 1.0 — promoted to front of scan queue - Tier C: excluded from shortlist

`reload_coin_tiers()` called after background reclassification completes.

Dynamic Exclusions

`auto_exclude_coin(symbol)` adds coins with insufficient data (<60 candles) to `data/coin_exclusions.json`. Persisted across restarts.

3.7 Risk Manager (risk_manager.py)

Risk Manager Routing

Segment	Risk Manager
L1	RM2_ATR
L2, AI, DePIN, Gaming, RWA, DeFi, Meme	RM3_Swing

`config.get_optimal_rm(symbol)` returns the RM ID for a given symbol.

Stop Loss / Take Profit Computation (`calculate_optimal_stops`)

RM2_ATR: - multiplier `m = 2.5` (3.5 for Meme/AI segments) - `SL = entry − direction × (m × ATR)` - `TP = entry + direction × (m × 2.0 × ATR)` → 1:2 R:R

RM3_Swing: - `SL = swing_l` (for BUY) or `swing_h` (for SELL) — 10-candle rolling local extremum - Falls back to `entry − direction × (3.0 × ATR)` if swing level is invalid - `TP = entry + direction × (risk_dist × 2.5)` → 2.5R R:R on actual swing risk

Leverage Selection

Two paths: 1. **Dynamic ATR-based** (`EXECUTION_DYNAMIC_LEVERAGE=True` , primary): linear scale from ATR% - ATR% ≤ 0.5% → 25x - ATR% ≥ 1.5% → 10x - Linear between [10x, 25x] 2. **Conviction-based** (legacy fallback): conviction score → leverage tier

Conviction Score (5 active factors, 0–100)

Factor	Weight	Source
HMM Confidence (margin tier)	71	`_score_hmm()`
BTC Macro Regime alignment	7	`_score_btc_macro()`
Funding Rate carry signal	11	`_score_funding()`
Open Interest Change	11	`_score_oi()`
Order Flow	0 (disabled)	—
SR + VWAP	0 (removed)	—
Sentiment	0 (removed)	—
Volatility quality	0 (removed)	—

Minimum conviction to proceed to Athena: 60 (`MIN_CONVICTION_FOR_DEPLOY`)

Kill Switch

`check_kill_switch()` triggers if portfolio drawdown ≥ 10% in 24h. Writes KILL command to `data/commands.json` , closes all positions.

Margin-First Position Sizing (`calculate_margin_first_position`)

```
For each leverage tier [35, 25, 15, 10, 5] (descending):
  If tier > conviction_leverage: skip
  sl_mult = get_atr_multipliers(lev)[0]
  loss_at_sl = (atr × sl_mult / price) × lev × 100  [% of margin]
  If loss_at_sl <= max_loss_pct: use this leverage
```

```
If final_leverage < MIN_LEVERAGE_FLOOR (5): skip trade
quantity = (margin × final_leverage) / price
```

3.8 Execution Engine (execution_engine.py)

Handles paper vs live order routing. Paper trades go to Binance testnet; live trades go to CoinDCX.

Limit order mechanics: - If `EXECUTION_ATR_PULLBACK=True` and `ema_15m_20` is available: places limit order at 20-EMA - Otherwise: market order - Virtual Ghost Limits (`EXECUTION_VIRTUAL_LIMITS=True`): limit orders held locally in tradebook as `status=OPEN`, not sent to exchange until triggered — prevents margin deadlock

SL/TP placement: - Calls `RiskManager.calculate_optimal_stops()` using coin's segment RM - Returns `(stop_loss, take_profit, rm_id)` passed to `tradebook.open_trade()` as `override_sl/override_tp`

3.9 Tradebook (tradebook.py)

Persistent trade journal in `data/tradebook.json`. Thread-safe via `_book_lock`.

Trade Status Lifecycle

```
OPEN    (limit order placed, awaiting fill)
  ↓ fill detected (paper: price crosses, live: virtual trigger)
ACTIVE  (position live, P&L tracked)
  ↓ exit condition met
CLOSED  (final P&L recorded)

Also:
CANCELLED  (limit order cancelled — TIF_EXPIRED or ESCAPE_HATCH)
```

Key Functions

Function	Purpose
`open_trade(...)`	Record new entry; deduplicates by `symbol+profile_id`
`close_trade(trade_id, symbol, exit_price, reason)`	Record exit, compute P&L, fees
`cancel_trade(trade_id, reason)`	Cancel OPEN limit; no P&L recorded

Function	Purpose
`activate_limit_order(trade_id, fill_price, fill_qty)`	OPEN → ACTIVE transition
`update_unrealized(prices, funding_rates)`	Batch P&L update + exit checks
`update_trade(trade_id, updates)`	Patch any trade fields
`get_active_trades()`	Returns list of ACTIVE + OPEN trades

P&L Computation

```
raw_pnl = (exit_price - entry_price) × quantity    [LONG]
raw_pnl = (entry_price - exit_price) × quantity    [SHORT]

Note: quantity is already leveraged (= capital × leverage / price)
DO NOT multiply by leverage again.

commission = (entry_notional + exit_notional) × TAKER_FEE (0.05%)
net_pnl = raw_pnl - commission - funding_cost
pnl_pct = net_pnl / capital × 100
```

Funding Rate Accumulation

Every heartbeat, `update_unrealized()` checks how many 8-hour intervals have elapsed since `last_funding_check`. Uses live CoinDCX funding rate if available, else `DEFAULT_FUNDING_RATE` (0.01%). Accumulates in `trade["funding_cost"]`.

Trailing SL — Stepped Profit Lock (`TRAILING_SL_STEPS`)

Every heartbeat, for each ACTIVE trade:

```
For each step (trigger_pnl_pct, lock_pnl_pct) in TRAILING_SL_STEPS:
  If pnl_pct >= trigger_pnl AND step_idx > stepped_lock_level:
    lock_price_move = (lock_pnl / 100) / leverage
    new_sl = entry × (1 + lock_price_move)   [LONG]
    new_sl = entry × (1 - lock_price_move)   [SHORT]
    If new_sl improves on trailing_sl: update it
    stepped_lock_level = step_idx
```

Steps defined in `TRAILING_SL_STEPS`: - At +5% P&L → SL to breakeven - At +10% → lock +5% - At +15% → lock +10% - ... continues to +50% → lock +45%

For live trades: calls `ExecutionEngine.modify_sl_live()` to update exchange SL order.

3.10 Athena LLM Layer (llm_reasoning.py)

File: `llm_reasoning.py` — class `AthenaEngine`

Model: `gemini-2.5-flash` via Google Generative AI REST API (`https://generativelanguage.googleapis.com/v1beta/models/...`)

Design: Fail-open. If API is unavailable, returns `AthenaDecision(action="EXECUTE", adjusted_confidence=1.0)` — trades are never blocked due to infrastructure failure.

Signal Context Sent to Athena

```
{
  "ticker":          symbol,           # e.g. "SOLUSDT"
  "side":            "BUY"/"SELL",
  "leverage":        int,
  "hmm_confidence": float,           # HMM margin (0.0–1.0)
  "hmm_regime":      str,             # e.g. "BULLISH"
  "conviction":      float,           # 0–100 multi-factor score
  "current_price":   float,
  "atr":             float,
  "atr_pct":         float,           # ATR as % of price
  "trend":           str,             # EMA20/50 trend direction
  "signal_type":     str,             # "TREND_FOLLOW" or "REVERSAL_PULLBACK"
  "ema_15m_20":      float,
  "tf_agreement":    int,             # 0–3 TFs agreeing
  "btc_regime":      str,             # from BTCUSDT coin state
}
```

Athena Prompt

Athena acts as Lead Investment Officer. Tasks per coin: 1. Technical price action (candles, momentum) 2. Support & Resistance levels 3. FVG and Order Blocks 4. Current news (Gemini web grounding) 5. BTC macro regime alignment 6. Final conviction: LONG / SHORT / SKIP 7. Leverage and position size recommendation

HMM signal carries 40% weight; Athena's own analysis carries 60%.

Decision Mapping

Athena Output	Engine Action
LONG or SHORT	EXECUTE
SKIP	VETO
adjusted_confidence < 0.30 (LLM_VETO_THRESHOLD)	VETO (forced)

Rate Limiting & Caching

- Max 5 API calls per cycle (`LLM_MAX_CALLS_PER_CYCLE`)
- Cache per coin for 10 minutes (`LLM_CACHE_MINUTES`)
- `reset_cycle()` called at start of each `_tick()`

Decision Logging

All non-cached decisions logged to `data/athena_decisions.json` (last 200 entries) and in-memory buffer (last 50).

`AthenaDecision` dataclass

```
@dataclass
class AthenaDecision:
    action: str              # EXECUTE, VETO, REDUCE_SIZE
    adjusted_confidence: float
    reasoning: str
    risk_flags: list
    athena_direction: str    # LONG, SHORT, SKIP (original)
    model: str
    latency_ms: int
    cached: bool
```

3.11 Engine API (engine_api.py)

Flask app wrapping the trading bot. Auth: Bearer token via `ENGINE_API_SECRET` env var. All routes require auth except `/api/health`.

Engine runs in background thread via `_run_engine()` with auto-restart: up to 5 retries with exponential backoff (10s base), then 5-minute recovery cooldown before infinite retry.

Engine API Endpoints (26 total)

Method	Endpoint	Purpose
GET	`/api/all`	Primary dashboard endpoint: multi_bot_state, tradebook, engine_state, segment_heatmap, athena state, registered_bot_ids
GET	`/api/health`	Engine status, uptime, cycle info, memory, crash history (no auth)

Method	Endpoint	Purpose
GET	`/api/gemini-health`	Validate Gemini API key connectivity
POST	`/api/close-trade`	Close single trade (live: CoinDCX first, then tradebook)
POST	`/api/close-all`	Queue CLOSE_ALL command
POST	`/api/exit-all-live`	Immediately close all CoinDCX positions + tradebook (called on bot stop)
POST	`/api/reset-trades`	Clear all tradebook entries (paper mode only)
POST	`/api/sync-exchange`	Reconcile tradebook with CoinDCX positions
POST	`/api/set-mode`	Switch paper/live mode at runtime
GET	`/api/validate-exchange`	Test exchange API key; returns balance
POST	`/api/set-config`	Apply per-bot risk config (max_loss_pct, capital_per_trade, max_open_trades)
POST	`/api/set-bot-id`	Register bot with engine (adds to ENGINE_ACTIVE_BOTS)
POST	`/api/remove-bot-id`	Deregister bot from ENGINE_ACTIVE_BOTS
POST	`/api/restart`	Force-restart engine thread
POST	`/api/resume`	Clear paused/halted state
GET	`/api/broadcast-log`	Last N lines of signal_broadcast.log as structured JSON
POST	`/api/pause`	Pause engine (optional halt_until timestamp)

Method	Endpoint	Purpose
GET	`/api/scanner`	Latest coin scanner results
GET	`/api/athena`	Athena state + recent decisions
POST	`/api/force-cycle`	Trigger immediate analysis cycle
GET	`/api/multi-state`	Raw multi_bot_state.json
GET	`/api/trade-history`	All tradebook entries
GET	`/api/segment-heatmap`	Latest segment heatmap
POST	`/api/kill-switch`	Trigger kill switch immediately
POST	`/api/reset-kill-switch`	Reset kill switch
GET	`/api/coin-state`	State for a specific coin

`/api/all` **response structure:**

```
{
  "multi": { "coin_states": {...}, "last_analysis_time": "...", "deployed_count": N
        },
  "tradebook": { "trades": [...], "summary": {...} },
  "engine": { "status": "running", ... },
  "heatmap": { "segments": [...] },
  "athena": { "enabled": true, "model": "gemini-2.5-flash", "recent_decisions":
        [...] },
  "registered_bot_ids": ["botId1", "botId2"]
}
```

---

4. Signal Pipeline — End-to-End Flow

```
Every 15 minutes:

1. SCAN POOL
   get_active_bot_segment_pool(ENGINE_ACTIVE_BOTS)
   → BTC always first, then segment coins (exclusions applied)

2. FOR EACH COIN IN BATCH:
```

```
   a. fetch_klines(symbol, "1h", 250)            [1h data for legacy brain]
   b. For each TF in ["1d", "1h", "15m"]:
      - fetch_klines(symbol, tf, 300)
      - compute_all_features(df_tf)
      - HMMBrain.train(df_tf)  [if needs_retrain()]
      - MultiTFHMMBrain.set_brain(tf, brain)
      - tf_data[tf] = df_tf_feat

   c. MultiTFHMMBrain.predict(tf_data)
   d. conviction, side, tf_agreement = mtf_brain.get_conviction()
      → If side=None (no consensus): return None

   e. Macro Veto: if side=BUY and BTC dropped >1.5% in 15m: skip

   f. Volatility filter: ATR% must be in [0.3%, 6%]

   g. Conviction threshold: if conviction < 60: skip

   h. Fetch 15m data for ema_15m_20 (limit order target)

   i. Return raw_result dict: {symbol, side, conviction, confidence, atr,
ema_15m_20, ...}

3. SORT raw_results by conviction DESC

4. FOR EACH BOT in ENGINE_ACTIVE_BOTS:
   a. Filter raw_results to bot's segment
   b. Pick top 1 coin (TOP_COINS_PER_SEGMENT=1)
   c. Check duplicate: if bot_id:symbol in active tradebook → skip
   d. ATHENA CALL:
      - Build llm_ctx with ticker, side, hmm_confidence, conviction, btc_regime,
etc.
      - AthenaEngine.validate_signal(llm_ctx)
      - If action=VETO: skip
      - If API fails: auto-EXECUTE (fail-open)
   e. EXECUTE:
      - ExecutionEngine.execute_trade(symbol, side, leverage, quantity, atr,
ema_15m_20, ...)
      - → CoinDCX limit/market order (live) or Binance testnet (paper)
   f. RECORD:
      - tradebook.open_trade(..., bot_id=bot_id, all_bot_ids=[bot_id], ...)
      - _active_positions[f"{bot_id}:{symbol}"] = {...}
```

---

5. Exit Mechanics

All exits except MAX_LOSS are handled in `tradebook.update_unrealized()` (called every 10s heartbeat). For live trades, CoinDCX handles SL/TP at the exchange level; tradebook mirrors state via `_sync_coindcx_positions()`.

Exit Trigger	Threshold	Notes
MAX_LOSS	`pnl_pct <= -35%` (configurable via `set-config`)	Hard stop. Fires BEFORE fixed SL for deep SELL trades where EMA20 SL is above market (SELL enters above EMA20, SL = EMA20 above entry → hit requires -45%+ move, MAX_LOSS catches at -35% first). This is by design, not a bug.
Fixed SL / TP	`current_price <= trailing_sl` (LONG) or `>= trailing_sl` (SHORT)	Uses trailing_sl value (which starts at fixed SL and improves as TRAILING_SL_STEPS activate)
Trailing SL Steps	+5% P&L → breakeven; +10% → lock +5%; etc.	10-step ladder up to +50% trigger → +45% lock. Modifies `trailing_sl` in-place
Escape Hatch	Price moves > `2.0 × ATR` from OPEN limit order entry price	Cancels OPEN limit order: `status=CANCELLED`, P&L=0. Prevents stale limit orders from filling far from intended price
TIF Expiry	OPEN limit order age > 60 minutes (`EXECUTION_TIF_MINUTES`)	Cancels OPEN limit order: `status=CANCELLED`, P&L=0
Regime-change exits	DISABLED	`_check_exits()` only syncs `_active_positions` dict; no regime-based forced closes. Backtest showed regime exits hurt returns

Exit Trigger	Threshold	Notes
Kill Switch	Portfolio drawdown ≥ 10% in 24h	Closes all positions, halts new deployments
KILL_SWITCH command	Manual trigger from dashboard	Same as kill switch

73% early exit root cause (documented): SELL trades entered when EMA20 is above market price. RM2_ATR places SL at EMA20 above entry. For SL to be hit, price would need to rally 45%+ from entry. MAX_LOSS at -35% fires first. This is correct behavior — MAX_LOSS acts as a wider safety net for trades where the structural SL is effectively out of reach.

6. Risk Management

Leverage Tiers

Conviction Score	Leverage
< 60	0 (no trade)
60–69	15x
70–94	25x
≥ 95	35x

Dynamic ATR leverage overrides conviction tiers when `EXECUTION_DYNAMIC_LEVERAGE=True` (default):

ATR%	Leverage
≤ 0.5%	25x
≥ 1.5%	10x
Between	Linear interpolation

Capital Allocation

- `CAPITAL_PER_TRADE = $100` per trade (fixed)
- `PAPER_MAX_CAPITAL = $2,500` total portfolio (25 slots × $100)
- Live mode: caps margin at `min($100, balance × 5%)` via `CAPITAL_PER_COIN_PCT`

ATR Multipliers by Leverage

```
def get_atr_multipliers(leverage):
    if leverage >= 50: return (0.5, 1.0)
    elif leverage >= 10: return (1.0, 2.0)   # 1:2 R:R — backtest-proven
    elif leverage >= 5:  return (1.2, 2.4)
    else:                return (1.5, 3.0)
```

Key Risk Parameters

Parameter	Value	Effect
`MAX_LOSS_PER_TRADE_PCT`	-35%	Hard stop on leveraged P&L%
`MIN_LEVERAGE_FLOOR`	5x	Skip trade if risk-capped leverage drops below 5x
`KILL_SWITCH_DRAWDOWN`	10%	Portfolio drawdown threshold for kill switch
`RISK_PER_TRADE`	4%	Risk % used in `calculate_position_size()`
`MIN_CONVICTION_FOR_DEPLOY`	60	Minimum conviction before Athena call
`VOL_MIN_ATR_PCT`	0.3%	Minimum volatility to trade
`VOL_MAX_ATR_PCT`	6.0%	Maximum volatility to trade
`MACRO_VETO_BTC_DROP_PCT`	1.5%	BTC flash crash threshold to block BUY signals

7. Crypto Segments

Segment	Coins	Risk Manager
L1	BTCUSDT, ETHUSDT, SOLUSDT, BNBUSDT, AVAXUSDT, SUIUSDT	RM2_ATR
L2	ARBUSDT, OPUSDT,	RM3_Swing

Segment	Coins	Risk Manager
	POLUSDT, MNTUSDT, STRKUSDT, IMXUSDT, RONINUSDT	
DeFi	UNIUSDT, AAVEUSDT, CRVUSDT, JUPUSDT, RUNEUSDT, PENDLEUSDT, LINKUSDT	RM3_Swing
AI	TAOUSDT, INJUSDT, WLDUSDT	RM3_Swing
Meme	DOGEUSDT, SHIBUSDT, PEPEUSDT, BONKUSDT	RM3_Swing
RWA	ONDOUSDT, POLYXUSDT, TRUUSDT	RM3_Swing
Gaming	AXSUSDT, SANDUSDT, PIXELUSDT, IOTXUSDT	RM3_Swing
DePIN	ARUSDT, HNTUSDT	RM3_Swing
Modular	TIAUSDT, DYMUSDT	RM3_Swing
Oracles	PYTHUSDT, TRBUSDT, API3USDT	RM3_Swing

Excluded globally: `AKTUSDT, FETUSDT, WIFUSDT, FILUSDT` — removed from all segment lists at import time.

Static exchange exclusions (scanner level): `EURUSDT, WBTCUSDT, USDCUSDT, TUSDUSDT, BUSDUSDT, USTUSDT, DAIUSDT, FDUSDUSDT, CVCUSDT, USD1USDT`

Segment rotation: `SCANNER_SEGMENT_ROTATION=True`. Updates master shortlist every 1 hour; determines active segments from 15-minute time block within the hour. `MAX_ACTIVE_PER_SEGMENT=1` limits correlation.

8. SaaS Dashboard (NextJS)

8.1 Technology Stack

Layer	Technology
Framework	Next.js 14 (App Router)
Language	TypeScript
Styling	Tailwind CSS + shadcn/ui components
ORM	Prisma
Database	PostgreSQL (Railway)
Auth	NextAuth.js (credentials provider)
Payments	Razorpay webhooks
Deployment	Railway (Docker)

8.2 Application Pages

Route	Description
`/`	Landing page (marketing)
`/dashboard`	Main dashboard: BTC regime, coin states, active trades, segment heatmap, bot cards
`/tradebook`	Full trade history with P&L charts, filter by bot/status
`/settings`	API key management, bot configuration
`/admin`	Admin panel: all users, subscription management, engine diagnostics

8.3 API Routes — Complete List (44 routes)

Bot Management:

Route	Method	Purpose
`/api/bots/create`	POST	Create new bot record in DB
`/api/bots/toggle`	POST	Start/stop bot: registers with engine, validates exchange, opens/closes sessions
`/api/bots/kill`	POST	Emergency kill: closes all trades, stops bot
`/api/bots/retire`	POST	Archive/retire bot
`/api/bots/config`	POST	Update bot configuration
`/api/bots/delete`	POST	Delete bot and all associated records
`/api/bots/logs`	GET	Fetch bot logs
`/api/bots/broadcast-log`	GET	Signal broadcast audit log from engine

Engine State:

Route	Method	Purpose
`/api/bot-state`	GET	Primary dashboard data: engine state, coin states, trades, heatmap, per-bot stats
`/api/health`	GET	NextJS app health check
`/api/engine-logs`	GET	Engine log tail
`/api/engine-debug`	GET	Engine debug diagnostics
`/api/cycle-snapshot`	POST	Receive per-cycle signal archive from engine; persists to DB
`/api/debug`	GET	Debug info

Trades:

Route	Method	Purpose
`/api/trades`	GET	List user trades from DB

Route	Method	Purpose
`/api/trades/close`	POST	Close specific trade (proxies to engine)
`/api/trades/exit-all`	POST	Close all active trades
`/api/trades/sync`	POST	Manually reconcile engine trades → Prisma
`/api/reset-trades`	POST	Clear all trades (paper mode)

Auth & Users:

Route	Method	Purpose
`/api/auth/[...nextauth]`	ALL	NextAuth handler
`/api/signup`	POST	User registration
`/api/sessions`	GET	Bot session list
`/api/sessions/backfill`	POST	Backfill session statistics

Subscription:

Route	Method	Purpose
`/api/subscription/status`	GET	Current subscription tier and status
`/api/webhooks/razorpay`	POST	Payment webhook handler

Admin:

Route	Method	Purpose
`/api/admin/users`	GET	All users list
`/api/admin/bots`	GET	All bots across users
`/api/admin/stats`	GET	Platform-wide statistics
`/api/admin/audit`	GET	Audit log
`/api/admin/subscriptions/change`	POST	Modify user subscription
`/api/admin/engine`	GET/POST	Engine admin controls

Route	Method	Purpose
`/api/admin/cleanup-trades`	POST	Cleanup orphaned trades
`/api/admin/close-engine-trade`	POST	Force-close specific engine trade
`/api/admin/mark-trades-closed`	POST	Bulk mark trades as closed
`/api/admin/reset-engine-tradebook`	POST	Reset engine tradebook
`/api/admin/trade-timeline`	GET	Trade timeline view
`/api/admin/orchestrator/health`	GET	Orchestrator health
`/api/admin/orchestrator/control`	POST	Orchestrator control

Market & Exchange:

Route	Method	Purpose
`/api/live-market`	GET	Live market prices
`/api/exchange/validate`	GET	Validate exchange API keys
`/api/exchange/positions`	GET	Live exchange positions
`/api/wallet-balance`	GET	Exchange wallet balance
`/api/performance`	GET	Bot performance metrics
`/api/settings/api-keys`	POST/GET	Manage exchange API keys (encrypted)

8.4 Bot Lifecycle

```
1. CREATE:    POST /api/bots/create
              → Bot record in DB (status="stopped", isActive=false)

2. CONFIGURE: POST /api/bots/config
              → BotConfig record: mode, segment, capitalPerTrade, maxLossPct, etc.

3. ACTIVATE:  POST /api/bots/toggle { botId, isActive: true }
              a. Subscription gate check (free tier can't start live bots)
              b. Exchange validation (live mode: test CoinDCX keys)
              c. POST /api/set-bot-id to engine → adds to ENGINE_ACTIVE_BOTS
              d. POST /api/set-config to engine → max_loss_pct, capital_per_trade
```

```
                e. POST /api/set-mode if live → switches engine to live mode
                f. createBotSession() in DB
                g. Bot.isActive=true, status="running", startedAt=now

4. TRADING:     Engine picks bot's segment, stamps trades with bot_id
                bot-state GET syncs trades engine → Prisma every 10s (dashboard poll)

5. DEACTIVATE: POST /api/bots/toggle { botId, isActive: false }
                a. POST /api/remove-bot-id from engine
                b. Live mode: POST /api/exit-all-live → close CoinDCX positions
                c. Close Prisma trades (if exchange close succeeded)
                d. closeBotSession() in DB
                e. POST /api/set-mode paper (revert engine)
                f. Bot.isActive=false, status="stopped", stoppedAt=now
```

8.5 Auto Re-Registration

After engine restart, `ENGINE_ACTIVE_BOTS` is cleared (in-memory). On every `GET /api/bot-state`:

```
For each active bot in DB:
  modeKey = bot's mode (live or paper)
  engineData = already-fetched engine response for that mode

  // Guard: skip ONLY if engine is unreachable (null)
  // Do NOT skip if registered_bot_ids is empty — that's post-restart state
  if engineDataByMode[modeKey] === null: skip   // engine unreachable
  if engineDataByMode[modeKey] === undefined: skip  // not fetched

  if bot.id NOT in registered_bot_ids:
    POST /api/set-bot-id to engine  // re-register
```

This guard is critical: `registered_bot_ids=[]` (empty list) means the engine restarted and needs re-registration. `null` means engine is offline and we shouldn't attempt registration.

For mixed-mode users (some bots live, some paper), both engines are fetched in parallel and re-registration is done per-engine.

8.6 Trade Sync Flow

Every `GET /api/bot-state` call:

1. Determine engine mode (live/paper) based on user's active bots
2. Fetch engine data from primary engine; if mixed modes, fetch both engines
3. For each user bot with `startedAt` set:
    - Pull engine tradebook trades matching bot's mode from cache
    - Call `syncEngineTrades(botTrades, bot.id, bot.startedAt)` → upsert to Prisma Trade table

4. Fetch user's trades from Prisma via `getUserTrades(userId)`
5. Enrich with live engine data (unrealized P&L not stored in Prisma)

8.7 BTC Confidence Display

The dashboard displays BTC "confidence" as a 0–100 integer. BTC's raw HMM margin is near-zero (CHOP/high-volatility state conviction is low). Instead:

```
// Regime string format: "1d=BULLISH(0.45) | 1h=BEARISH(0.32) |
        15m=SIDEWAYS/CHOP(0.08)"
const matches = regimeStr.match(/\(([\d.]+)\)/g)
// Extract: [0.45, 0.32, 0.08]
const avg = values.reduce((a, b) => a + b) / values.length
return Math.round(avg * 100)   // → e.g. 28
```

Fallback chain: conviction → raw margin → 0.

---

9. Data Routing — End-to-End

Engine → Dashboard (Read path)

```
Engine writes:          data/multi_bot_state.json
                        data/tradebook.json
                        data/segment_heatmap.json
                        data/athena_decisions.json
                        data/signal_broadcast.log

bot-state/route.ts:    GET → engine /api/all (every dashboard poll)
                        → Merge with Prisma trades
                        → Return unified JSON to dashboard-client.tsx

dashboard-client.tsx:   Renders BTC regime panel, coin heatmap,
                        active trades list, segment heatmap cards,
                        bot status cards, Athena decision feed
```

Trade Recording (Write path)

```
Engine:                 tradebook.open_trade() → data/tradebook.json

bot-state/route.ts:    syncEngineTrades() → Prisma Trade.upsert()
                        (keyed by trade_id field)
```

```
dashboard-client.tsx:   Displays from Prisma (via getUserTrades)
                        Active trades show unrealized P&L from engine
```

Bot Registration Flow

```
User clicks "Start":   bots/toggle POST → validates subscription
                        → engine /api/set-bot-id (adds to ENGINE_ACTIVE_BOTS)
                        → engine /api/set-config (max_loss, capital)
                        → DB: bot.isActive=true

Engine _tick():        reads config.ENGINE_ACTIVE_BOTS
                        → scans segment for each bot
                        → stamps trades with bot_id

bot-state GET:         checks registered_bot_ids from engine response
                        → if bot missing: auto re-register
```

Cycle Snapshot (Audit path)

```
After each _tick():    Engine POSTs to dashboard /api/cycle-snapshot
                        → Prisma CycleSnapshot, CoinScanResult, SegmentHeatmapEntry
                        (background thread, non-blocking, 10s timeout)
```

---

10. Deployment & Infrastructure

Platform: Railway (Docker containers)

Engine service: `engine_api.py` is the Flask entrypoint. `main.py` (RegimeMasterBot) runs in a background thread started at Flask app init.

SaaS service: NextJS app in `sentinel-saas/nextjs_space/`.

Two-engine setup: Separate Railway services for paper and live trading. Environment variable `ENGINE_URL_PAPER` and `ENGINE_URL_LIVE` control which engine each user's bots connect to (set in Railway env vars for the NextJS service).

Auto-restart: Engine thread has 5-retry exponential backoff + infinite recovery loop. SIGTERM handler ensures graceful shutdown on Railway redeploy.

Key environment variables:

Variable	Service	Purpose
`BINANCE_API_KEY/SECRET`	Engine	Paper trading data
`COINDCX_API_KEY/SECRET`	Engine	Live order execution
`GEMINI_API_KEY`	Engine	Athena LLM
`ENGINE_API_SECRET`	Engine	Bearer token auth for all engine endpoints
`ALLOW_LIVE_TRADING`	Engine	Safety guard; must be `true` to enable live mode
`PAPER_TRADE`	Engine	Default mode (overridden at runtime by set-mode)
`ENGINE_BOT_ID`	Engine	Default bot ID (overridden by set-bot-id)
`ENGINE_BOT_NAME`	Engine	Default bot name
`DATABASE_URL`	SaaS	PostgreSQL connection string
`NEXTAUTH_SECRET`	SaaS	Session encryption
`ENGINE_URL_PAPER`	SaaS	Paper engine Flask URL
`ENGINE_URL_LIVE`	SaaS	Live engine Flask URL
`ORCHESTRATOR_URL`	SaaS	Orchestrator service URL (default http://localhost:5000)
`ENGINE_INTERNAL_SECRET`	SaaS/Engine	Secret for cycle-snapshot POST auth
`TELEGRAM_BOT_TOKEN/CHAT_ID`	Engine	Telegram alerts (TELEGRAM_ENABLED=False by default)

11. Configuration Reference

Key parameters from `config.py` and their effect:

Trading Loop

Parameter	Default	Effect
`LOOP_INTERVAL_SECONDS`	10	Heartbeat interval
`ANALYSIS_INTERVAL_SECONDS`	900	Full analysis cycle (15 minutes)
`SCAN_INTERVAL_CYCLES`	4	Rebuild coin pool every N cycles (1 hour)
`TOP_COINS_LIMIT`	50	Max coins in scan pool
`MAX_CONCURRENT_POSITIONS`	10	Max simultaneous open positions

Multi-TF HMM

Parameter	Default	Effect
`MULTI_TF_ENABLED`	True	Use 3-TF aggregation (disable for legacy 1H-only)
`MULTI_TF_TIMEFRAMES`	["1d","1h","15m"]	Timeframes per coin
`MULTI_TF_CANDLE_LIMIT`	300	Candles fetched per TF
`MULTI_TF_WEIGHTS`	{1d:40, 1h:35, 15m:25}	Conviction weighting
`MULTI_TF_MIN_AGREEMENT`	2	Minimum TFs agreeing to produce signal
`HMM_N_STATES`	3	Bull/Bear/Chop (4-state CRASH removed — merged into BEAR)
`HMM_RETRAIN_HOURS`	24	Model retraining frequency

Segment Scanner

Parameter	Default	Effect
`SCANNER_SEGMENT_ROTATION`	True	Enable 15-min block rotation
`SEGMENT_SCAN_LIMIT`	3	Top N segments selected by heatmap
`SCANNER_COINS_PER_SEGMENT`	5	Max coins per segment in shortlist

Parameter	Default	Effect
`MAX_ACTIVE_PER_SEGMENT`	1	Correlation control

Conviction & Deploy

Parameter	Default	Effect
`MIN_CONVICTION_FOR_DEPLOY`	60	Gate before Athena call
`TOP_COINS_PER_SEGMENT`	1	Athena evaluates top N per segment per bot
`CONVICTION_WEIGHT_HMM`	71	HMM factor weight (out of 100)
`CONVICTION_WEIGHT_FUNDING`	11	Funding rate factor weight
`CONVICTION_WEIGHT_OI`	11	OI change factor weight
`CONVICTION_WEIGHT_BTC_MACRO`	7	BTC macro alignment factor

Athena

Parameter	Default	Effect
`LLM_REASONING_ENABLED`	True	Enable Athena layer
`LLM_MODEL`	gemini-2.5-flash	Gemini model
`LLM_CACHE_MINUTES`	10	Cache decision per coin
`LLM_MAX_CALLS_PER_CYCLE`	5	Rate limit per analysis cycle
`LLM_VETO_THRESHOLD`	0.30	Confidence below this → auto-VETO
`LLM_TIMEOUT_SECONDS`	30	API call timeout

Risk / Exit

Parameter	Default	Effect
`CAPITAL_PER_TRADE`	100	USD per trade (fixed)

Parameter	Default	Effect
`MAX_LOSS_PER_TRADE_PCT`	-35	Hard stop (leveraged P&L%)
`TRAILING_SL_ENABLED`	True	Enable stepped profit lock
`TRAILING_SL_STEPS`	10-step ladder	First step: +5% trigger → breakeven
`EXECUTION_ESCAPE_ATR`	2.0	Cancel pending limit if price moves >2 ATR
`EXECUTION_TIF_MINUTES`	60	Pending limit order lifetime
`EXECUTION_ATR_PULLBACK`	True	Place limit at 20-EMA instead of market
`EXECUTION_DYNAMIC_LEVERAGE`	True	ATR%-based leverage (overrides conviction tiers)
`EXECUTION_MAX_LEVERAGE`	25	Dynamic leverage ceiling
`EXECUTION_MIN_LEVERAGE`	10	Dynamic leverage floor

Disabled Features (in config, not active)

Feature	Status	Config flag
Sentiment Engine (FinBERT/VADER)	DISABLED	`SENTIMENT_ENABLED=False`
Order Flow Engine (L2 depth)	DISABLED	`ORDERFLOW_ENABLED=False`
Multi-Target Partial Profit	DISABLED	`MULTI_TARGET_ENABLED=False`
Weekend Skip	DISABLED	`WEEKEND_SKIP_ENABLED=False`
Regime-Change Exits	DISABLED	Code in `_check_exits()` is a no-op

Prisma Database Schema Summary

Model	Key Fields	Purpose
`User`	id, email, password, role	User accounts
`Subscription`	userId, tier, status, trialEndsAt	Subscription gating
`ExchangeApiKey`	userId, exchange, apiKey (encrypted)	Per-user exchange credentials
`Bot`	userId, name, exchange, isActive, startedAt	Bot instances
`BotConfig`	botId, mode, capitalPerTrade, maxLossPct, segment, brainType	Bot settings
`BotState`	botId, engineStatus, lastCycleAt, coinStates	Engine state mirror
`BotSession`	botId, startedAt, endedAt, totalTrades, totalPnl	Trading session records
`Trade`	botId, coin, position, entryPrice, stopLoss, takeProfit, status, totalPnl	Individual trades
`PartialBooking`	tradeId, target, exitPrice, pnl	Partial profit bookings
`CycleSnapshot`	cycleNumber, btcRegime, coinsScanned, deployedCount	Per-cycle audit records
`CoinScanResult`	cycleId, symbol, regime, conviction, deployStatus	Per-coin scan results
`SegmentHeatmapEntry`	cycleId, segment, compositeScore, isSelected	Heatmap per cycle

Trade field `bot_id` (snake_case in engine JSON) maps to Prisma `Trade.botId` during sync. The sync key is `trade_id` (engine) → matched against existing Prisma records for upsert.